Lucrare de Laborator 2

Procesarea Semnalelor

Elaborat: Macrii Danu
Gr.: FAF-222

Chișinău, March 2025

Contents

1 Generating Finite Sequences

1.1 Generating Two Finite Sequences

Generate two sequences $a(n)$ and $b(n)$ of length 5 and 4, respectively:

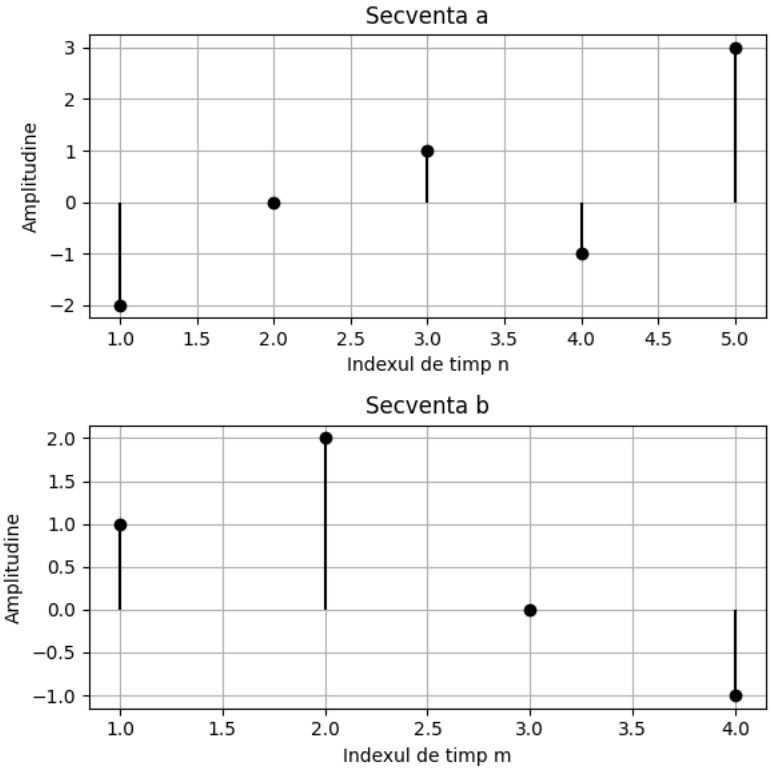
$$a = [-2, 0, 1, -1, 3] \; b = [1, 2, 0, -1]$$

Code:

```
a = np.array([-2, 0, 1, -1, 3])
b = np.array([1, 2, 0, -1])

n = np.arange(1, len(a) + 1)
m = np.arange(1, len(b) + 1)


fig, axs = plt.subplots(2, 1, figsize=(6, 6))
```

Plot the sequences:

- $d = 5$, $n = 1 : 1 : d$ for sequence a.
- $c = 4$, $l = 1 : 1 : c$ for sequence b.



2 Convolution of Two Sequences

Perform the convolution using the built-in function:

$$c = \text{conv}(a, b) \tag{1}$$

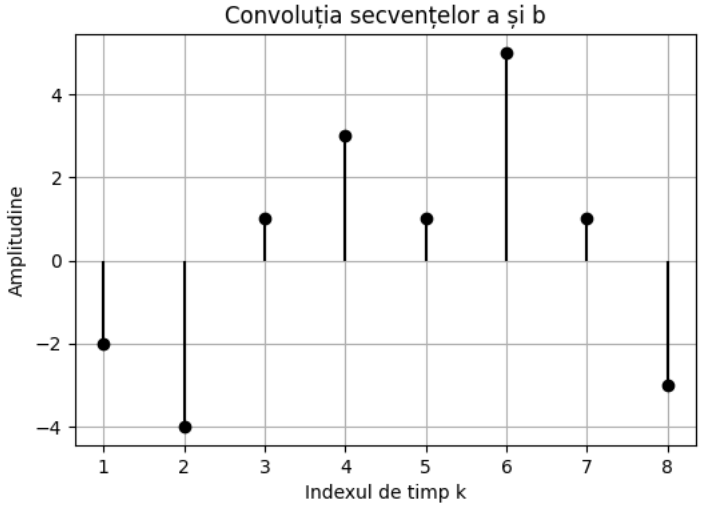
Define the convolution length as:

$$k = 1 : 1 : 8, \quad m = 8 \tag{2}$$

Code:

```
c = np.convolve(a, b, mode='full')
```

Plot the discrete convolution signal using stem(k, c).



3 Fourier Transform of Sequences

Determine the Fourier Transform of the sequences:

$$AE = \text{fft}(a, m) \; BE \qquad = \text{fft}(b, m)$$

Compute the product of the Fourier transforms:

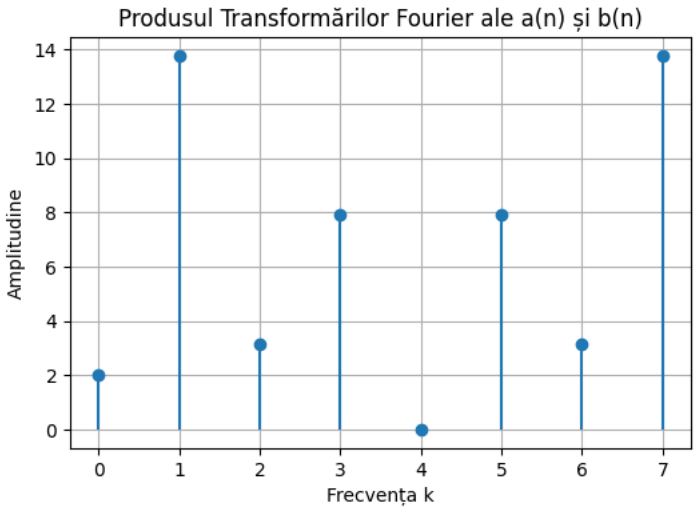
$$p = AE \cdot BE \tag{3}$$

This product is equal to the Fourier Transform of the convolution of $a(n)$ and $b(n)$.

Code:

```
a = np.array([-2, 0, 1, -1, 3])
b = np.array([1, 2, 0, -1])

m = len(a) + len(b) - 1

AE = np.fft.fft(a, m)
BE = np.fft.fft(b, m)
```

```
p = AE * BE

k = np.arange(m)
```



Plot stem(k, p).

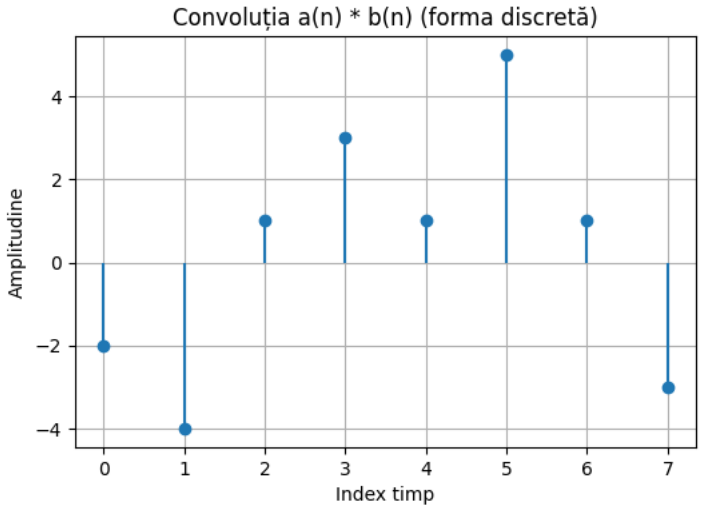
4 Inverse Fourier Transform of Product

Compute the inverse Fourier Transform:

$$y_1 = \text{ifft}(p) \tag{4}$$

Code:

```
y1 = np.fft.ifft(p)
```



Plot the discrete signal y_1.

5 Comparison of Convolution Results

Compare the computed convolution with the original result:

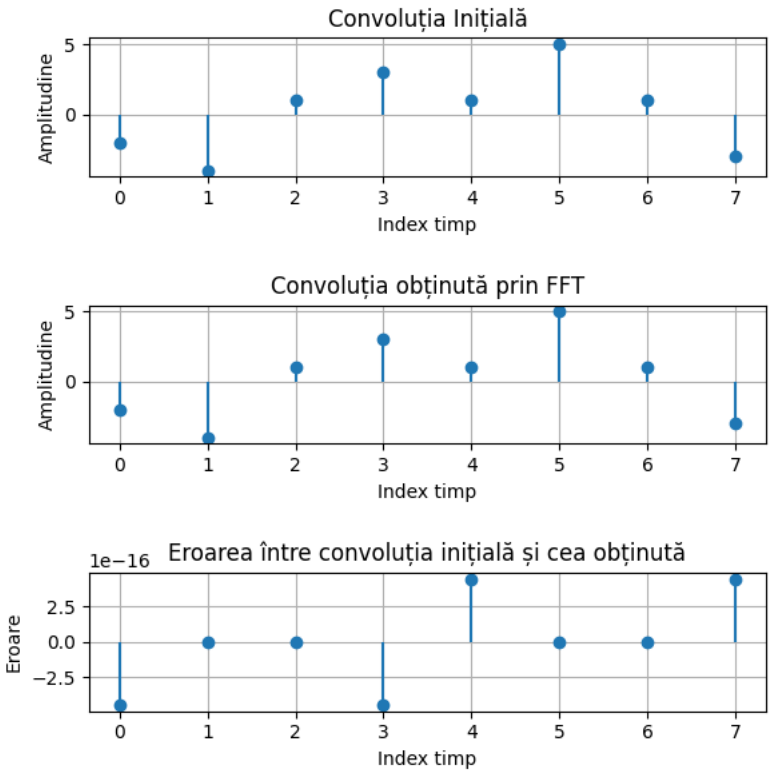
$$\text{error} = c - y_1 \tag{5}$$

Code:

```
c = np.convolve(a, b, mode='full')

m = len(a) + len(b) - 1

AE = np.fft.fft(a, m)
BE = np.fft.fft(b, m)

p = AE * BE

y1 = np.fft.ifft(p)

error = np.real(c[:m]) - np.real(y1)

k = np.arange(m)
```

Plot the signals in three separate subplots:

- subplot$(3, 1, 1)$: stem(k, c)
- subplot$(3, 1, 2)$: stem(k, y_1)
- subplot$(3, 1, 3)$: stem(k, error)



6 Repetition with Larger Sequences

Repeat steps 1-4 for significantly larger sequences to observe timing differences.

6.1 Generating Large Signals

Use known functions such as cosine, square, and sawtooth:

$$a = 2 \cdot \text{square}(20\pi n + 1) \quad b = 3 \cdot \text{sawtooth}(20\pi l + 1)$$

Code:

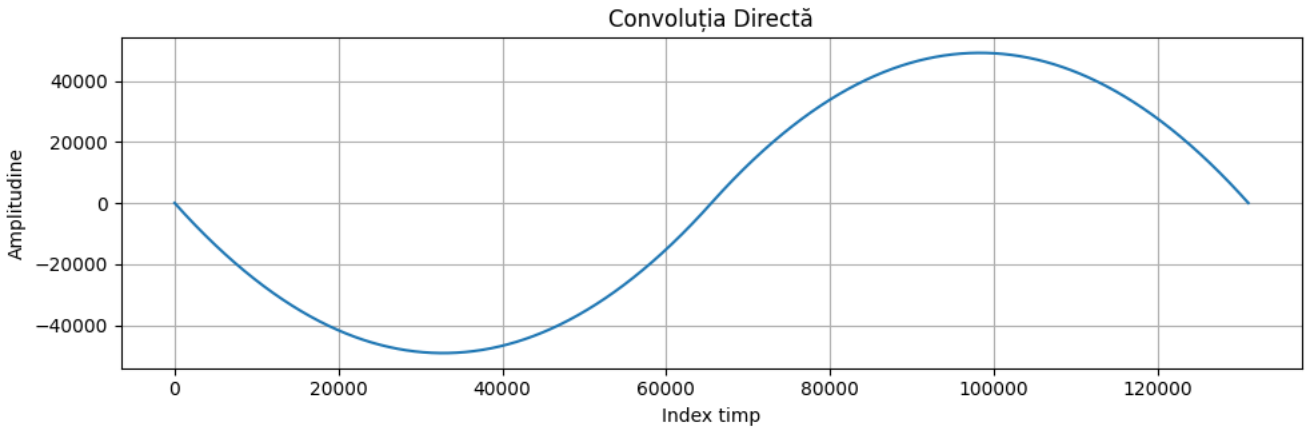
```
n = np.arange(0, 262144)
l = np.arange(0, 262144)

a = 2 * np.sign(np.sin(20 * np.pi * n + 1))
b = 3 * (np.mod(n / 65536, 1) - 0.5)

m = len(a) + len(b) - 1
m = 2 ** int(np.ceil(np.log2(m)))

AE = np.fft.fft(a, m)
BE = np.fft.fft(b, m)
p = AE * BE
y1 = np.fft.ifft(p)
time_convolution_fft = time.time() - start_time
```

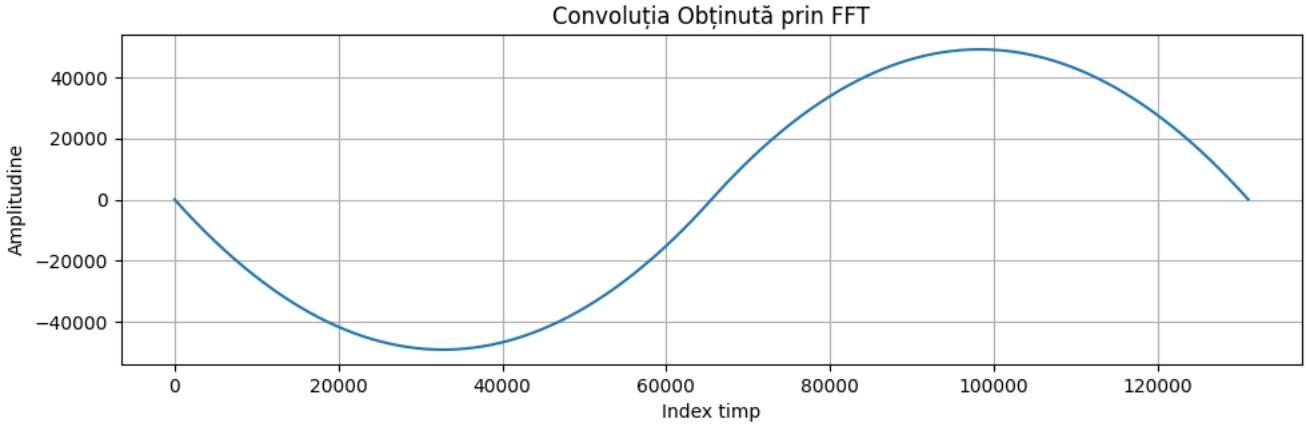
```
c_trimmed = c[:m]

error = np.real(c_trimmed) - np.real(y1[:len(c_trimmed)])

k = np.arange(len(c_trimmed))  # Adjust k to have the same length as c_trimmed and y1
```

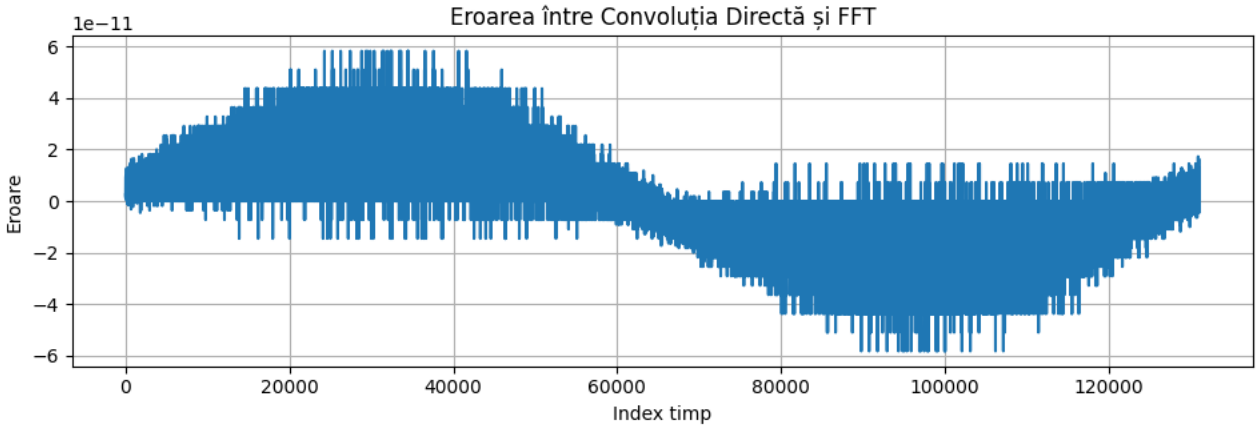
6.2 Computing Convolution Time

Measure execution time:

- Place tic before convolution computation.
- Place toc after convolution computation.







7 Computing FFT Convolution Time

Perform FFT-based convolution in one step and measure execution time:

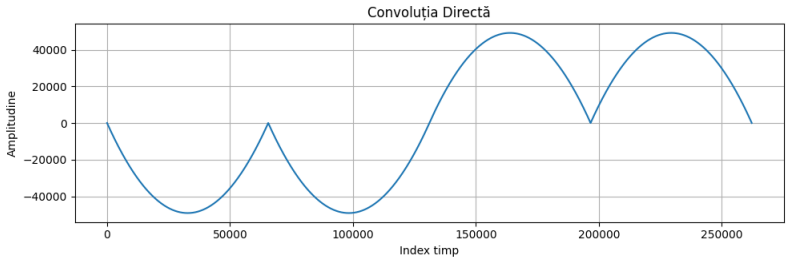
- Use tic before the first Fourier Transform.
- Use toc after the inverse Fourier Transform.

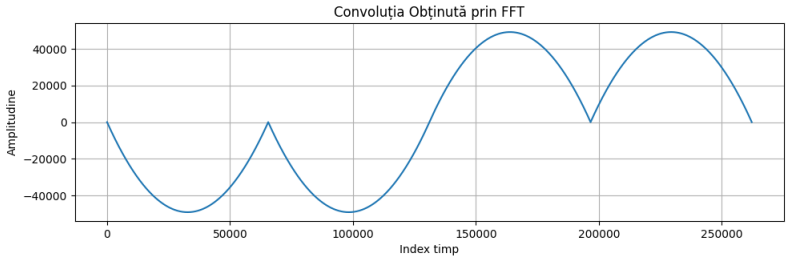
```
Timpul pentru convoluția directă: 1.095747 secunde
Timpul pentru convoluția cu FFT: 0.024653 secunde
Timpul total pentru convoluția directă: 1.095747 secunde
Timpul total pentru convoluția cu FFT: 0.024653 secunde
```

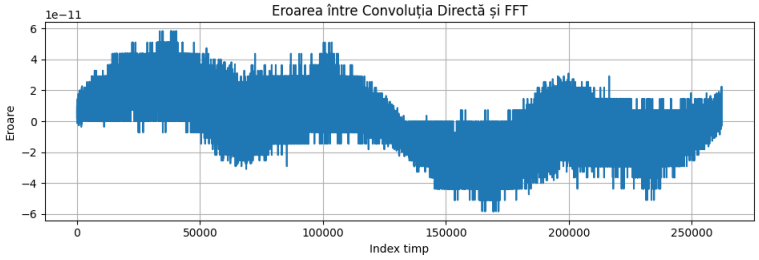
8 Repetition with Even Larger Sequences

Repeat the previous step with sequences of increasing length to analyze execution time variations. Example sequence lengths:

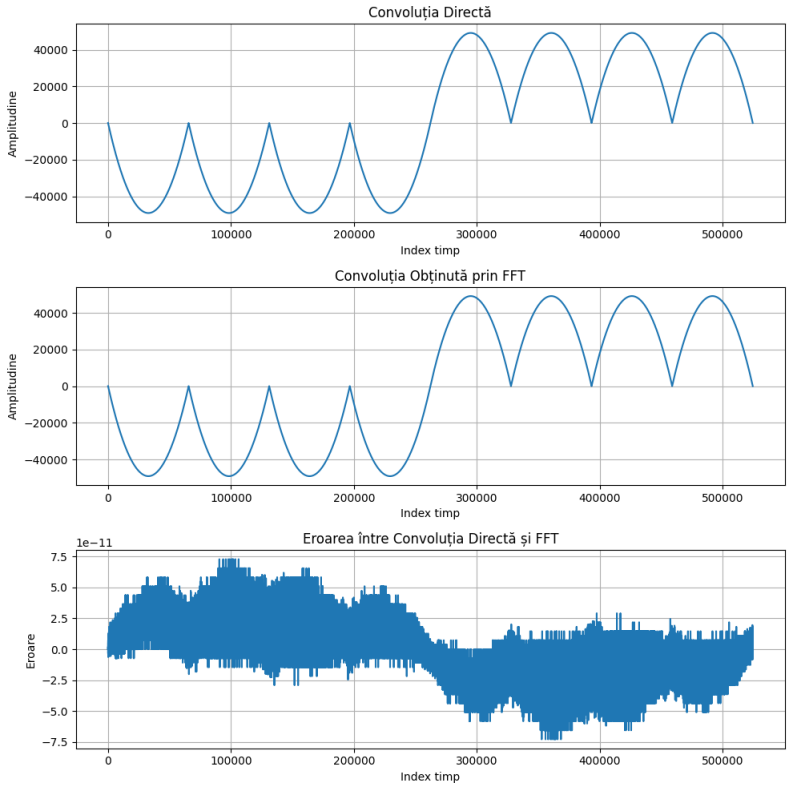
- $n = 0 : 1 : 131072$, $l = 1 : 1 : 131072$, $m = 262144$
- $n = 0 : 1 : 262144$, $l = 1 : 1 : 262144$, $m = 524288$
- $n = 0 : 1 : 524288$, $l = 1 : 1 : 524288$, $m = 1048576$








```
Timpul pentru convoluția directă: 4.010892 secunde
Timpul pentru convoluția cu FFT: 0.043926 secunde
Timpul total pentru convoluția directă: 4.010892 secunde
Timpul total pentru convoluția cu FFT: 0.043926 secunde

Process finished with exit code 0
```




```
Timpul pentru convoluția directă: 15.520268 secunde
Timpul pentru convoluția cu FFT: 0.085873 secunde
Timpul total pentru convoluția directă: 15.520268 secunde
Timpul total pentru convoluția cu FFT: 0.085873 secunde
```

9 Block-Wise Convolution

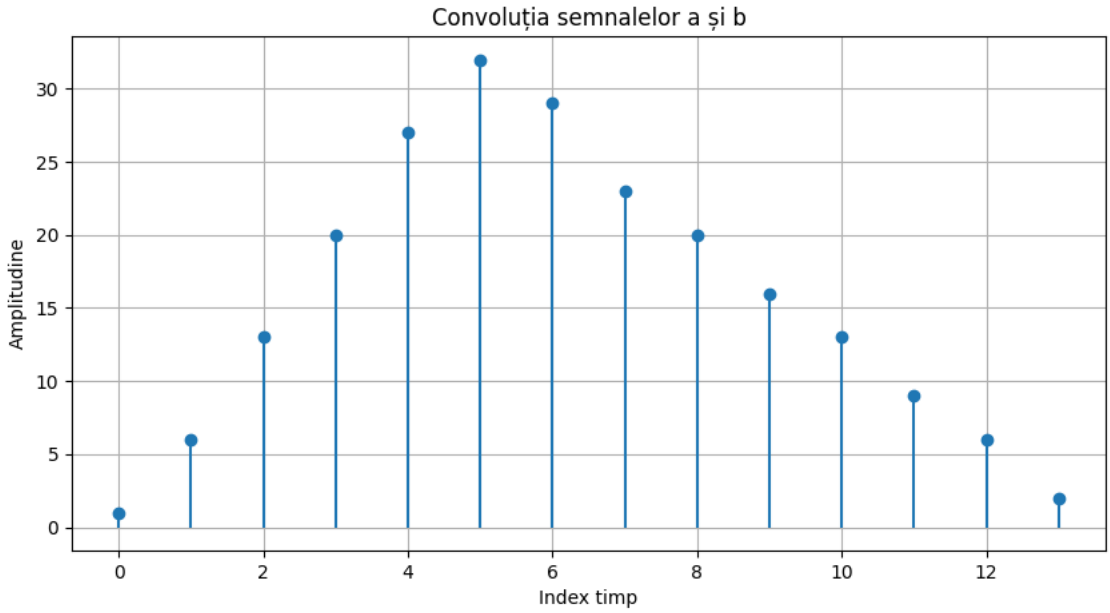
In some applications, convolution must be performed as the signal is received. In this case, block-wise convolution is applied. Given:

$$a = [1, 4, 2] \; b \qquad\qquad = [1, 2, 3, 4, 5, 4, 3, 3, 2, 2, 1, 1]$$

Compute the full convolution (length 14).

Code:

```
c = np.convolve(a, b, mode='full')
```



10 Block-Wise Processing

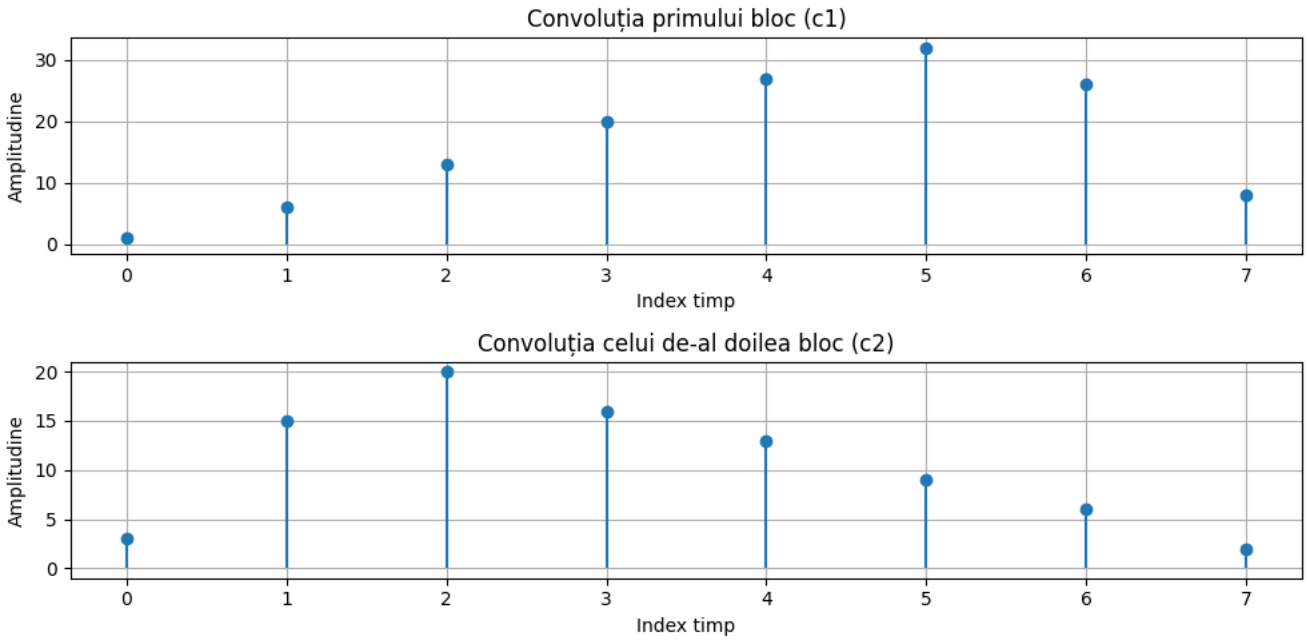
Divide the input signal into two blocks:

$$a_1 = a(1:6) \; a_2 \qquad\qquad = a(7:\text{end})$$

Compute the convolution of each block separately to obtain c_1 and c_2 (length 8 each).

Code:

```
a = np.array([1, 4, 2])
b = np.array([1, 2, 3, 4, 5, 4, 3, 3, 2, 2, 1, 1])

b1 = b[:6]
b2 = b[6:]

c1 = np.convolve(a, b1, mode='full')
c2 = np.convolve(a, b2, mode='full')

k1 = np.arange(len(c1))
k2 = np.arange(len(c2))
```



11 Final Convolution Reconstruction

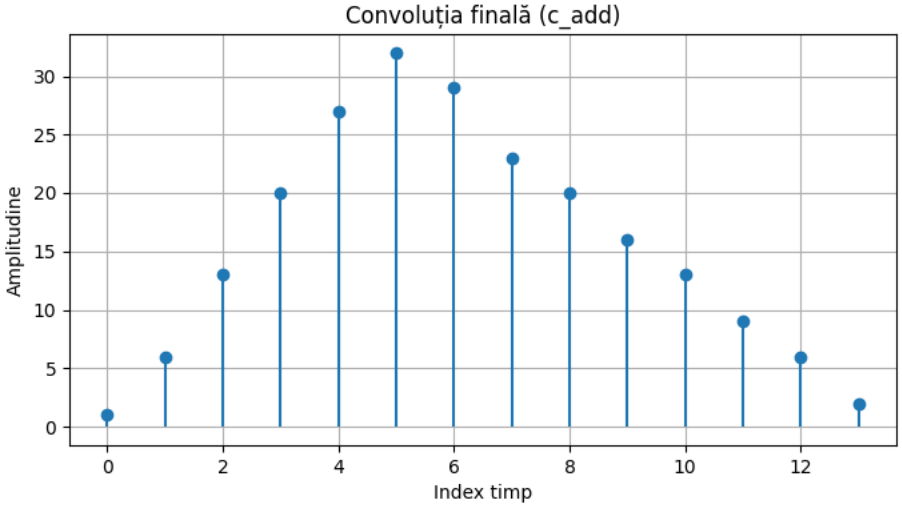
Construct the final convolution result:

$$c_{add} = [c_1(1:6), c_1(7:8) + c_2(1:2), c_2(3:\text{end})] \tag{6}$$

Code:

```
c_add = np.concatenate([c1[:6], c1[6:8] + c2[:2], c2[2:]])
```

Plot stem(m, c_{add}).



Conclusions

This laboratory work examined different approaches to computing the convolution of discrete signals, including direct convolution, Fourier Transform-based convolution, and block-wise convolution. The results demonstrated that while direct convolution provides exact results, the Fourier Transform method offers significant computational advantages for large signals due to its lower complexity. However, rounding errors and numerical inaccuracies were observed when using the inverse Fourier Transform. The block-wise convolution method illustrated a practical approach for real-time applications where signals are processed in chunks rather than as complete sequences. The time measurements confirmed that FFT-based convolution becomes increasingly efficient as signal length increases, making it a preferred method for large-scale signal processing. The experiment also highlighted the trade-offs between accuracy and computational efficiency in digital signal processing tasks. Future improvements could involve implementing optimized FFT algorithms and comparing performance across different hardware platforms.